

Programação de Sistemas

Revisão da Linguagem C Padrão - Módulo 5 -

Prof. Ronaldo Augusto de Lara Gonçalves

Agradecimentos ao Prof. Marcelo de G. Malheiros

- Um **procedimento** é um trecho de código com um identificador (nome), com ou sem passagem de parâmetros, que não retorna valor (tipo *void*), estruturado tal como o modelo a seguir:
 - `void identificador(parâmetros opcionais) {`
 - `bloco_de_comandos;`
 - `}`
- O corpo do procedimento (bloco de comandos) será executado quando ele for chamado pelo identificador, como se fosse um comando a ser executado.
- A chamada deve ser feita dentro da função *main()* ou dentro de outro procedimento ou função.
- A palavra *void* no local dos parâmetros indica o procedimento não utiliza parâmetros, podendo ser inclusive omitida. Colocada antes do identificador, a palavra *void* indica que não existe retorno de valor pelo identificador.

Procedimentos e Funções



Explicação

- No exemplo a seguir, o procedimento *hello()* é chamado para ser executado dentro da função *main()*, como se fosse um comando.
- Durante a execução dos comandos da função *main()*, quando o identificador *hello()* é encontrado, logo após a execução do **bloco de comandos do início**, o controle de execução (fluxo de execução) é desviado para dentro do bloco de comandos que está contido dentro do procedimento *hello()* e a instrução **`printf("Olá Mundo!\n")`** é executada;
- Após o término da execução do procedimento, o controle de execução volta novamente para dentro da função *main()*, para continuar a execução no comando seguinte ao procedimento chamado, ou seja, o **bloco de comandos do meio**.
- Logo depois, o procedimento **`imprime(msg1)`** é chamado para ser executado. Nesse caso, o endereço da *string* *msg1* é passado para o parâmetro *msg2*, e assim, a mensagem **`"Olá Mundo!\n"`** também é impressa. Após, o controle de execução retorna para o **bloco do fim** da função *main()*.
- São 2 formas de chamada de procedimento: sem e com parâmetros.

Procedimentos e Funções



Exemplo

```
void hello(void) {  
    printf("Olá Mundo!\n");  
}  
void imprime(char *msg2) {  
    printf("%s", msg2);  
}  
void main(void) {  
    char msg1[] = "Ola Mundo!\n";  
    bloco_do_inicio;  
    hello();  
    bloco_do_meio;  
    imprime(msg1);  
    bloco_do_fim;  
}
```

Procedimentos e Funções



» função, procedimento com retorno de valor

- Uma **função** é um procedimento que “retorna um valor no identificador”, conforme o tipo definido que o precede, tal como o seguinte exemplo, que retorna um número inteiro:
 - `int le_inteiro() {`
 - `int i;`
 - `printf("Entre com um inteiro: ");`
 - `scanf("%d", &i);`
 - `return i;`
 - `}`
- O valor é retornado pelo comando **return**. O valor retornado por uma chamada pode ser simplesmente mostrado na tela, passado como parâmetro ou atribuído a outra variável de mesmo tipo que a função, tal como os exemplos a seguir:
 - `printf("Numero lido = %d\n", le_inteiro());`
 - `int n = le_inteiro();`

» Parâmetros

- O comando *return* sempre finaliza a execução da função e o controle da execução retorna para o código chamador, para a instrução seguinte a chamada.
- Procedimento ou função pode ter um ou mais parâmetros, separados por vírgula, tal como segue
 - `float maior(float x, float y) {`
 - `float m = x;`
 - `if (y > m) {`
 - `m = y;`
 - `}`
 - `return m;`
 - `}`
- Parâmetros podem ser constantes (valores) ou variáveis de qualquer tipo ou ponteiros/endereços para qualquer tipo de alocação

Procedimentos e Funções



Exemplos Adicionais: Enviando dados básicos para o procedimento/função

```
○ void imprimeSoma(int x1, int x2) {  
    int soma;  
    ○ soma=x1+x2;  
    ○ printf("\nSoma de %d com %d = %d\n", x1, x2, soma);  
○ }  
○ void main(void) {  
    int A, B;  
    ○ printf("\nDigite A > ");  
    ○ scanf("%d",&A);  
    ○ printf("\nDigite B > ");  
    ○ scanf("%d", &B);  
    ○ imprimeSoma(A, B);  
    ○ imprimeSoma(5, 10);  
○ }
```

Procedimentos e Funções



Exemplos Adicionais: Enviando e recebendo dados básicos da função

```
○ int multiplica(int x1, int x2) {  
    int produto;  
    ○ produto=x1*x2;  
    ○ return produto;  
○ }  
○ void main(void) {  
    int A, B, M;  
    ○ printf("\nDigite A > ");  
    ○ scanf("%d",&A);  
    ○ printf("\nDigite B > ");  
    ○ scanf("%d", &B);  
    ○ printf("\nMultiplicacao: %d * %d = %d\n", A, B, multiplica(A,B));  
    ○ M=multiplica(5, 10);  
    ○ printf("\nMultiplicacao: %d * %d = %d\n", 5, 10, M);  
○ }
```


Procedimentos e Funções



Exemplos Adicionais: Enviando e recebendo dados básicos nos parâmetros

```
○ void subtrai(int x1, int x2, int *sb) {  
    ○ *sb=x1-x2;  
○ }  
○ void main(void) {  
    int A, B, S;  
    ○ printf("\nDigite A > ");  
    ○ scanf("%d",&A);  
    ○ printf("\nDigite B > ");  
    ○ scanf("%d",&B);  
    ○ subtrai(A, B, &S);  
    ○ printf("\nSubtracao: %d - %d = %d\n", A, B, S);  
    ○ subtrai(5, 10, &S);  
    ○ printf("\nSubtracao: %d - %d = %d\n", 5, 10, S);  
○ }
```

Procedimentos e Funções



Exemplos Adicionais: Enviando e recebendo vetores nos parâmetros

- ```
void leiaVetor(int *v, int tam) {
 int i;
 ○ for (i=0; i<tam; i++) {
 ○ printf("v[%d] = ", i);
 ○ scanf("%d", &v[i]);
 ○ }
○ }
```
- ```
void main(void) {  
    int vet[20];  
  
    ○ leiaVetor(vet, 5);  
    ○ imprimeVetor(vet, 5);  
○ }
```
- ```
void imprimeVetor(int *v, int tam) {
 int i;
 ○ for (i=0; i<tam; i++)
 ○ printf("v[%d] = %d\n", i, v[i]);
○ }
```

# Procedimentos e Funções



## Exemplos Adicionais: Enviando e recebendo strings nos parâmetros

- `void leiaNome(char *nome, int tam) {`
  - `printf("\nDigite Nome = ");`
  - `fgets(nome, tam, stdin);`
- `}`
- `ou`
- `void leiaNome(char nome[], int tam) {`
  - `printf("\nDigite Nome = ");`
  - `fgets(nome, tam, stdin);`
- `}`
- `void main(void) {`
  - `char nome[22];`
    - `leiaNome(nome, 20);`
    - `imprimeNome(nome);`
- `}`

- `void imprimeNome(char *nome) {`
  - `printf("\nNome = %s\n", nome);`
- `}`
- `ou`
- `void imprimeNome(char nome[]) {`
  - `printf("\nNome = %s\n", nome);`
- `}`
- `ou`
- `void main(void) {`
  - `char *nome;`
    - `nome=malloc(22*sizeof(char));`
  - `leiaNome(nome, 20);`
  - `imprimeNome(nome);`
- `}`

# Exercícios/Desafios – Parte 01



- Faça uma função para calcular fatorial de  $n$ , que tenha  $n$  como parâmetro e retorne o resultado no nome da função. Faça também um procedimento para imprimir a sequencia de Fibonacci de  $n$  elementos, sendo  $n$  passado como parâmetro. Faça também uma função para calcular a potencia de  $x$  elevado a  $n$ , sendo  $x$  e  $n$  passados como parâmetros, e o resultado retornado no nome da função. Faça também um procedimento para imprimir os  $n$  primeiros números primos, sendo  $n$  passado como parâmetro. Faça um programa principal que ofereça ao usuário um menu de 5 opções (1-Calcular Fatorial, 2-Imprimir Fibonacci, 3-Calcular Potência, 4-Imprimir Números Primos e 5-Sair do programa) para que ela escolha. O programa deve permitir reexecução, caso o usuário não queira sair do programa.
- Implemente uma segunda versão do código, mudando as funções para procedimento e os procedimentos para funções e ajustando o main. Entregue ambos os códigos no Moodle.

# Exercícios/Desafios – Parte 02



- Reimplemente o programa que calcula o produto escalar entre 2 vetores. O programa principal deverá chamar um procedimento e uma função: o procedimento será para gerar aleatoriamente um vetor e a função será para calcular o produto escalar. O procedimento deverá ter 2 parâmetros: o tamanho do vetor e o endereço do vetor. O procedimento deverá gerar os elementos e colocá-los na área de memória do vetor. O procedimento será chamado uma vez para cada vetor. A função terá 3 parâmetros (o tamanho dos vetores e os endereços dos 2 vetores) e deverá retornar o resultado do cálculo realizado no nome da função. O programa deve permitir reexecução, conforme interesse do usuário.
- Implemente uma segunda versão do código, mudando a função para procedimento e o procedimento para função e ajustando o programa principal. Entregue ambos os códigos no Moodle.

**Perguntas?**